USING MOSTEK'S F8 IN A SCANNED SEVEN-SEGMENT DISPLAY APPLICATION

Using Mostek's F8 In A Scanned Seven-Segment Display Application

by Dan Hammond

INTRODUCTION

Many microprocessor based devices require a numeric display as an integral part of the system. For reasons of cost and reliability, it is usually desirable to keep the chip count as low as possible with the microprocessor performing the control logic in software. Time multiplexed digit scanning is a common solution and works very well using a single F8 port for up to 8 digits.

THEORY OF OPERATION

An eight digit display can be scanned with one F8 port (fig. 1) by using half of the port for the BCD number and half for the digit select. When using the digit scanning technique an 'image' of the display must be maintained in memory, with a byte (or half byte) of memory containing the BCD number to be displayed in each of the eight digits. The following five steps show the basic control the software is required to execute:

Step 1 Output digit select and BCD number for this digit (from 'image')

Step 2 Turn on strobe

Step 3 Delay

Step 4 Turn off strobe

Step 5 Increment digit select, return to step 1

The scan rate should be fast enough to prevent the display from 'flickering'. It has been found that a 80 to 100Hz rate is sufficient for a stationary display. An approximate 100Hz rate is achieved in an eight digit display by making the delay in step 3, 1.25 milliseconds.

Maximum brilliance will be provided by leaving the strobe on for the whole delay time. This provides a 1/8 or 12.5% duty cycle. Reducing the strobe width will reduce the duty cycle and cause the display to be dimmer.

Interdigit blanking to prevent a blurring effect is accomplished by strobing the digit decoder after digit select/BCD number data is present on the port and removing the strobe before changing the data (see fig. 2)

EXAMPLE HARDWARE DESIGN

The example design in Figure 3 shows the hardware simplicity in an LED display scanning circuit interfaced to the F8. Bits 0-2 are used to select the digit, bit 3 as a strobe and bits 4-7 for the BCD number.

In this eight digit display the current required from the segment drivers and anode drivers is approximately 6-8 times what it would be for a static non-scanned display of equal brillance because only one digit is receiving current at a time (12.5% Duty Cycle).

F8 Family

NUMERIC DISPLAY BLOCK DIAGRAM

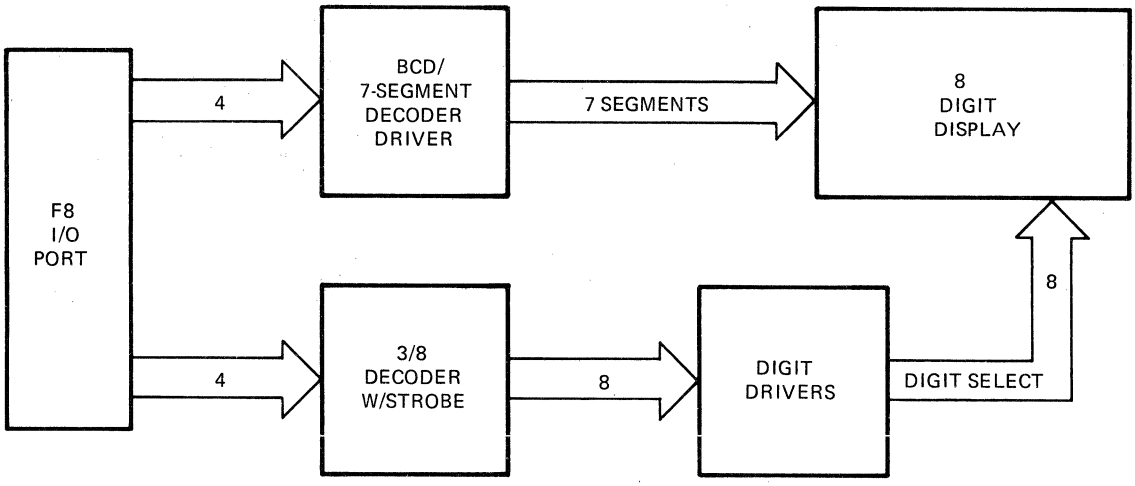

Figure 1

INTERDIGIT BLANKING

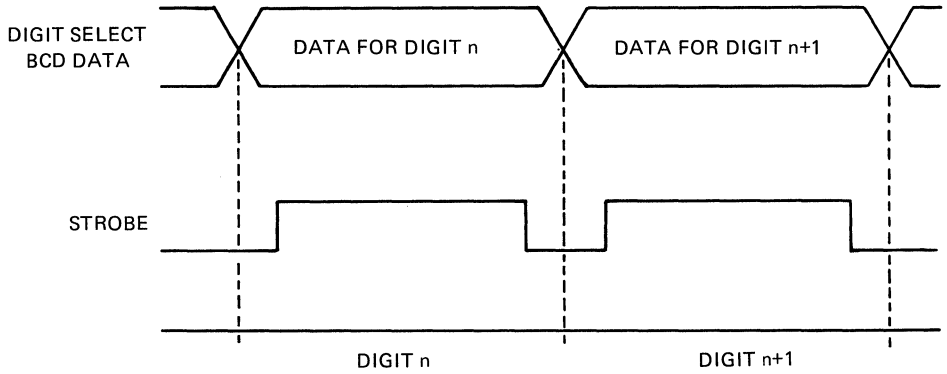

Figure 2

The SN7447 seven segment decoder/driver sinks 40mA per segment which will supply a maximum average current of 5mA per segment to each digit. This is an acceptable current level for many 7 segment LED displays such as the .43 inch HP7650. Since the anode transistors must drive seven segments, they will be required to source 280mA peak at a 12.5% duty cycle. Many discrete transistors (such as the 2N2907) and transistor arrays will handle this load.

7 SEGMENT DISPLAY INTERFACE

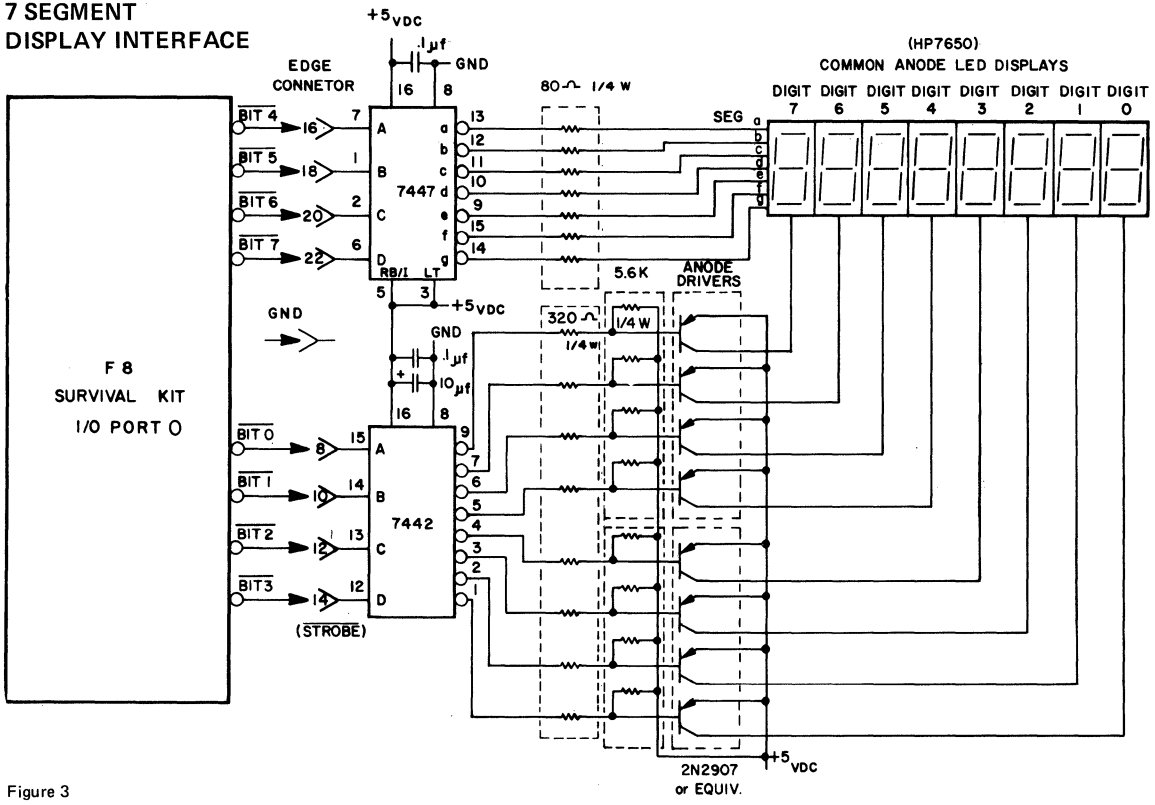

Figure 3

F8 Family

MAIN PROGRAM

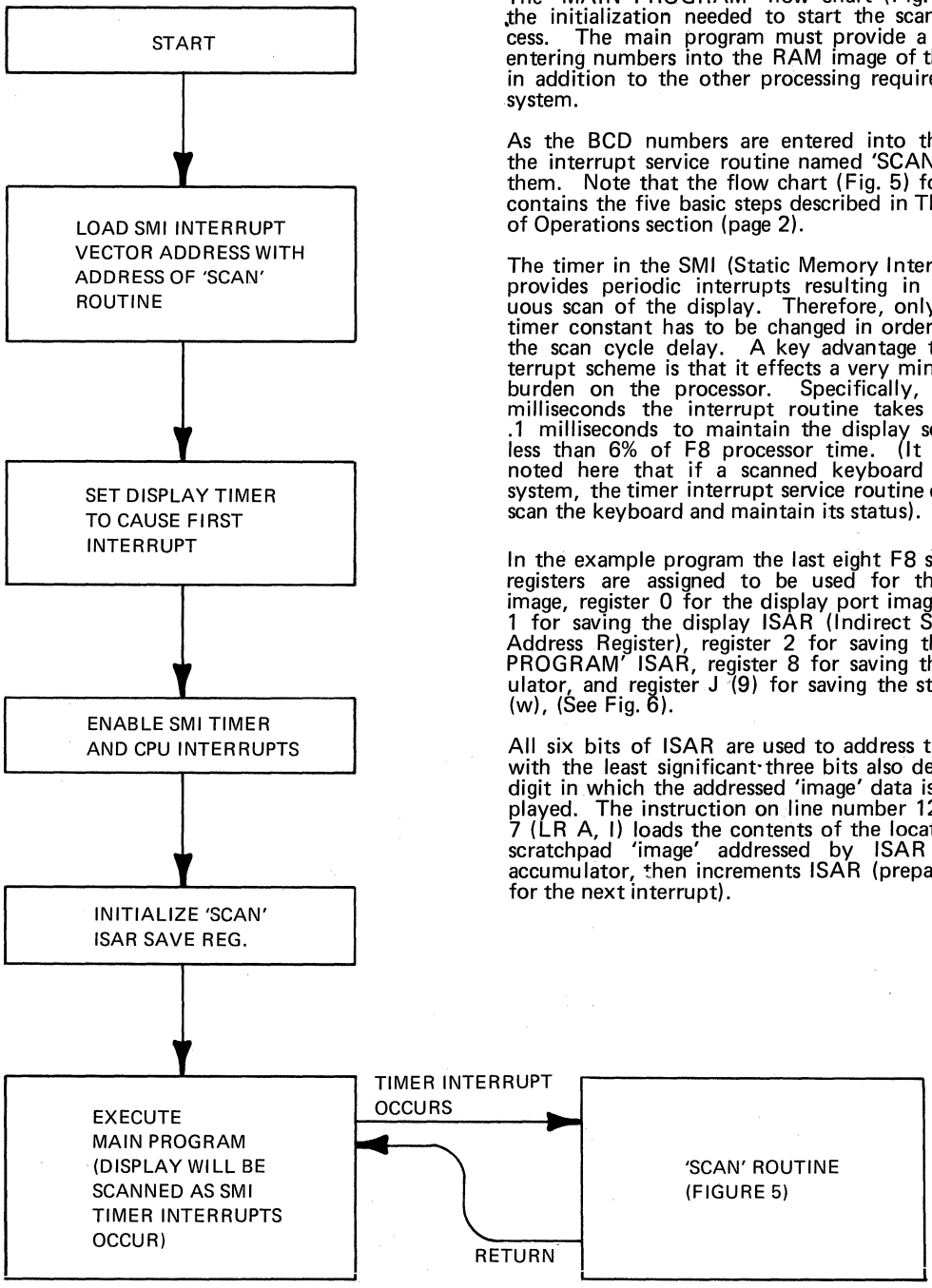

Figure 4

EXAMPLE CONTROL SOFTWARE

The 'MAIN PROGRAM' flow chart (Fig. 4) shows the initialization needed to start the scanning process. The main program must provide a means of entering numbers into the RAM image of the display in addition to the other processing required by the system.

As the BCD numbers are entered into the 'image' the interrupt service routine named 'SCAN' displays them. Note that the flow chart (Fig. 5) for 'SCAN' contains the five basic steps described in The Theory of Operations section (page 2).

The timer in the SMI (Static Memory Interface) chip provides periodic interrupts resulting in a continuous scan of the display. Therefore, only the SMI timer constant has to be changed in order to adjust the scan cycle delay. A key advantage to this interrupt scheme is that it effects a very minimal time burden on the processor. Specifically, every 1.5 milliseconds the interrupt routine takes less than .1 milliseconds to maintain the display scan, using less than 6% of F8 processor time. (It should be noted here that if a scanned keyboard is in the system, the timer interrupt service routine could also scan the keyboard and maintain its status).

In the example program the last eight F8 scratchpad registers are assigned to be used for the display image, register 0 for the display port image, register 1 for saving the display ISAR (Indirect Scratchpad Address Register), register 2 for saving the 'MAIN PROGRAM' ISAR, register 8 for saving the accumulator, and register J (9) for saving the status word (w), (See Fig. 6).

All six bits of ISAR are used to address the 'image' with the least significant three bits also defining the digit in which the addressed 'image' data is to be displayed. The instruction on line number 12 of figure 7 (LR A, I) loads the contents of the location in the scratchpad 'image' addressed by ISAR into the accumulator, then increments ISAR (preparing ISAR for the next interrupt).

INTERRUPT SERVICE ROUTINE FOR NUMERIC DISPLAY

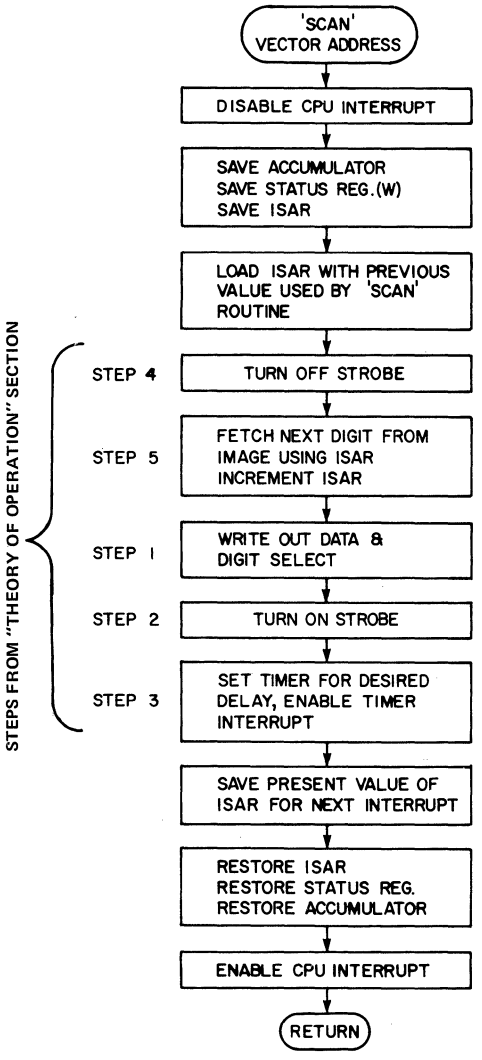

Figure 5

F8 SCRATCHPAD REGISTER USAGE MAP

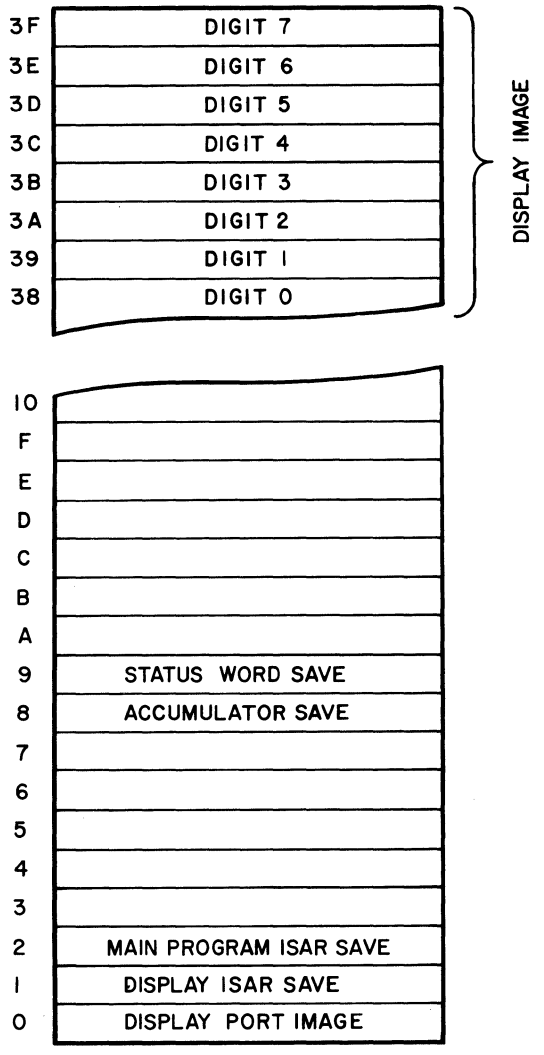

Figure 6

Output port H'F' is the timer constant register in the SMI chip (see line 1C in figure 7). Port H'E' is a register used to enable the timer interrupt in the SMI (line 1F). Note also that all outputs to the display port are 'OUTS 0' selecting port 0 (line E, line 17 & line 19).

The program listing (Fig. 7) contains comments that specify the purpose of each instruction.

SDB RESIDENT ASSEMBLER LISTING Figure 7

```
                OBJECT
LINE # ADDRESS  CODE                SOURCE CODE        COMMENTS
0000                     *                             INTERRUPT SERVICE ROUTINE
0001                     *                             FOR NUMERIC DISPLAY
0002                     *
0003                     *
0004                                ORG   H'700'
0005 0700 1A             SCAN       DI                 DISABLE CPU INTERRUPTS
0006 0701 58                        LR    8,A          SAVE ACCUMULATOR
0007 0702 1E                        LR    J,W          SAVE STATUS REG
0008 0703 0A                        LR    A,IS         LOAD ISAR INTO ACCUMLATOR
0009 0704 52                        LR    2,A          SAVE ISAR FROM MAIN PROGRAM
000A 0705 41                        LR    A,1          LOAD ACCUMULATOR WITH PREV ISAR
000B 0706 0B                        LR    IS,A         LOAD ISAR FOR SCAN
000C 0707 40                        LR    A,0          LOAD PREVIOUS DISPLAY PORT DATA
000D 0708 21 F7                     NI    H'F7'        MASK OUT STROBE BIT
000E 070A B0                        OUTS  0            TURN OFF STROBE
000F 070B 0A                        LR    A,IS         LOAD ISAR INTO ACCUMULATOR
0010 070C 21 07                     NI    7            MASK OUT ISAR(U)
0011 070E 50                        LR    0,A          ISAR(L) TO R0 FOR DIGIT # SELECT
0012 070F 4D                        LR    A,I          GET BCD DATA USING ISAR,INC ISAR
0013 0710 15                        SL    4            MOVE IT TO MS HALF OF ACCUMULATO
0014 0711 C0                        AS    0            ADD DIGIT # TO BCD DATA
0015 0712 18                        COM                INVERT DATA SINCE PORTS NEG TRUE
0016 0713 21 F7                     NI    H'F7'        MASK OUT STROBE BIT
0017 0715 B0                        OUTS  0            WRITE NEW DATA OUT (NO STROBE)
0018 0716 22 08                     OI    8            STROBE BIT ON
0019 0718 B0                        OUTS  0            TURN ON STROBE
001A 0719 50                        LR    0,A          SAVE DISPLAY PORT DATA
001B 071A 20 C4                     LI    H'C4'        TIMER CONSTANT
001C 071C BF                        OUTS  H'F'         WRITE TO SMI TIMER
001D 071D 73                        LIS   3            LOCAL INTERRUPT ENABLE BITS
001E 071E BE                        OUTS  H'E'         ENABLE LOCAL INTERRUPTS
001F 071F 0A                        LR    A,IS         LOAD ISAR INTO ACCUMULATOR
0020 0720 51                        LR    1,A          SAVE DISPLAY SCAN ISAR
0021 0721 42                        LR    A,2          LOAD MAIN PROGRAM ISAR VALUE
0022 0722 0B                        LR    IS,A         RESTORE ISAR WITH IT
0023 0723 1D                        LR    W,J          RESTORE STATUS REG
0024 0724 48                        LR    A,8          RESTORE ACCUMULATOR
0025 0725 1B                        EI                 ENABLE CPU INTERRUPTS
0026 0726 1C                        POP                RETURN TO MAIN PROGRAM
0027                                END
00

SCAN 0700
```

ALTERNATE DESIGN APPROACHES

There are several other approaches to a numeric display interface with the F8. For example, the BCD to seven segment conversion and 3/8 digit decoding could be done in software. This approach (Fig. 8) uses two ports.

If four ports are available, the display could also be driven statically, with each port controlling two digits. This approach (Fig. 9) would require one BCD to 7-segment decoder/driver (and 7 resistors) for each digit.

The best design approach depends on the application and the number of F8 ports available.

ALTERNATE SCANNING APPROACH

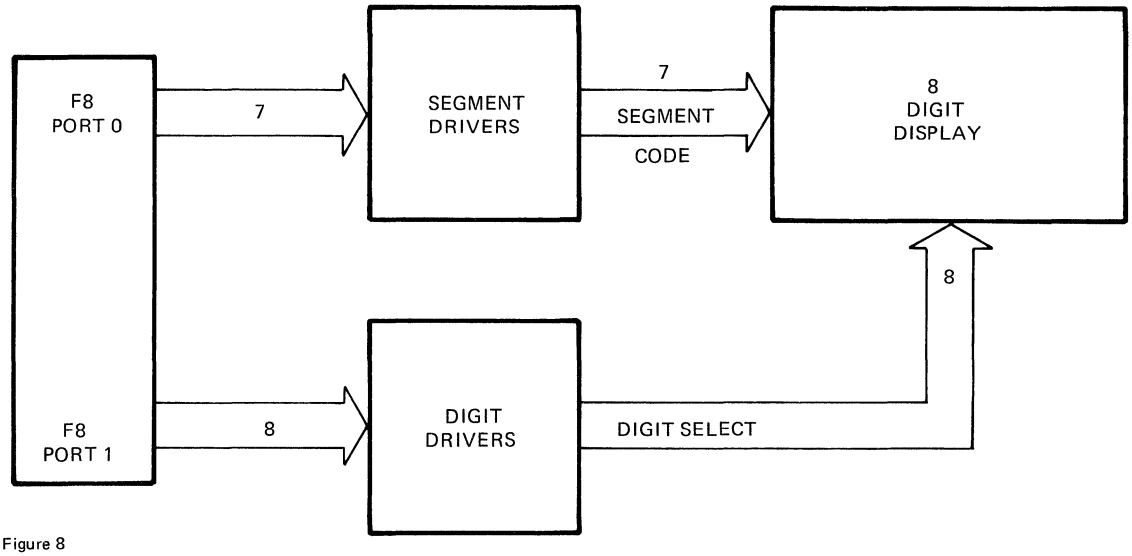

Figure 8

STATIC DISPLAY APPROACH

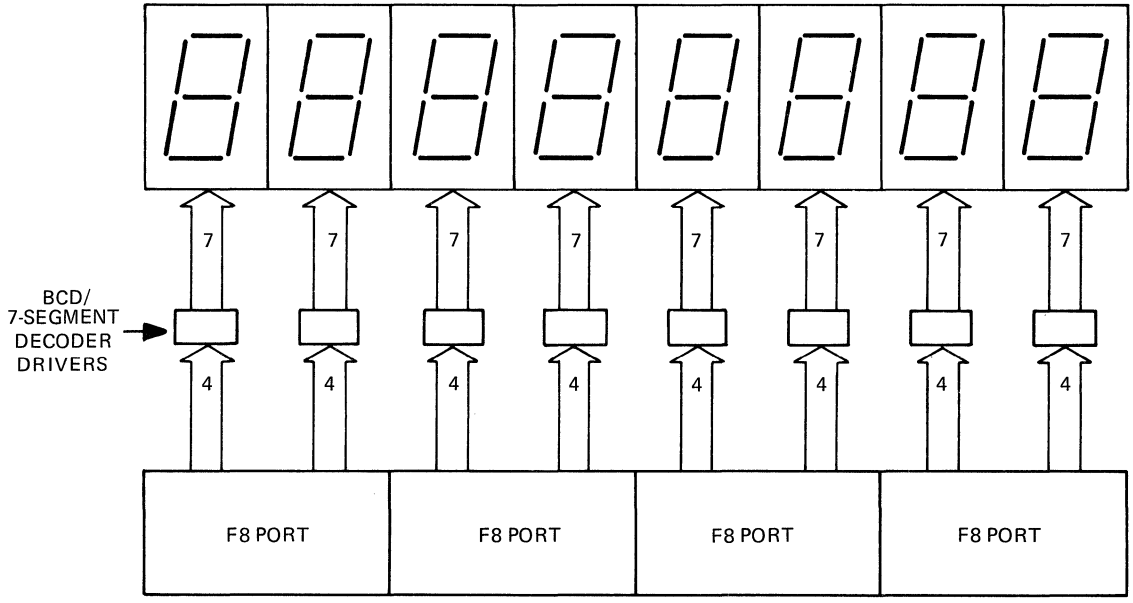

Figure 9

F8 Family